AWS API Gateway Lab Part 1

To Begin with the Lab

Summary of the Lab

This lab built a CRUD backend using **Amazon DynamoDB** and **AWS Lambda**. First, a UsersTable table was created with userId as the partition key, then an **IAM** role named LambdaDynamoDBExecutionRole was prepared so Lambda could access DynamoDB. After that, three separate Lambda functions were created: CreateUserFunction, GetUserFunction, and DeleteUserFunction, each using Python 3.12 and the same execution role. The create function was tested by inserting a sample user record, the get function was tested for both existing and missing users, and the delete function was tested by removing the record and confirming it disappeared from the table. This prepared the backend for the next step, which was API Gateway integration.

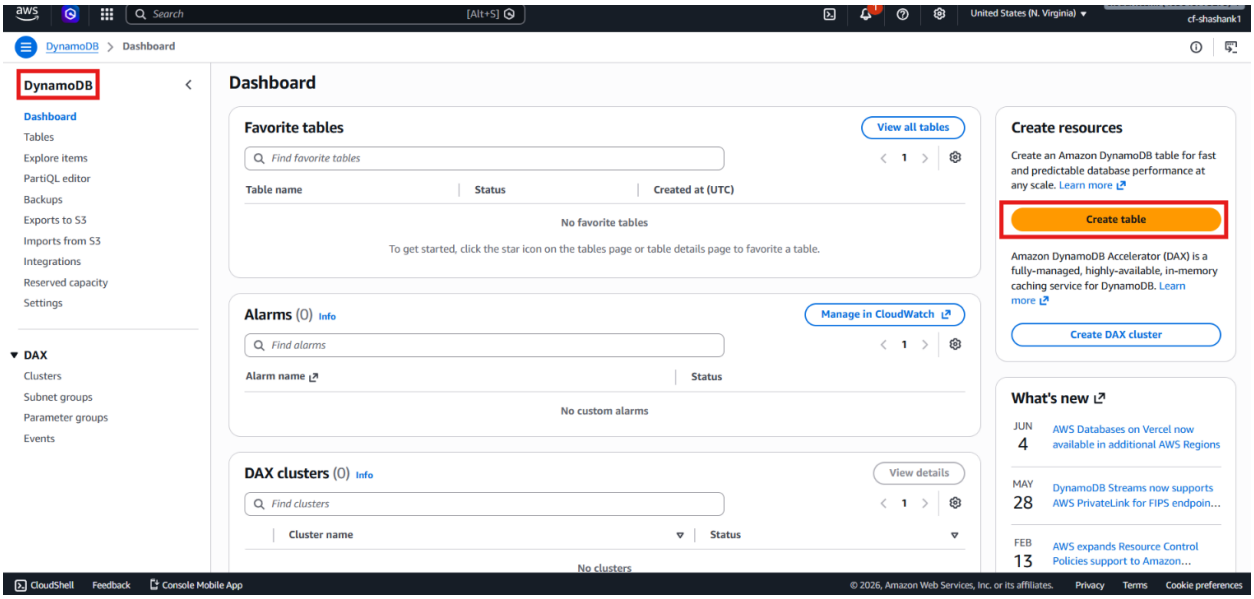
Understanding the Lab

This lab shows how to build the backend layer for a simple user-management application using **AWS Lambda** and **Amazon DynamoDB**. A front-end application should not talk directly to DynamoDB, so the logic is split into separate Lambda functions for create, read, and delete operations. Using one dedicated IAM role for all three functions keeps permissions controlled and reusable. This pattern is a clean way to implement separation of concerns and makes the backend easier to test and maintain.

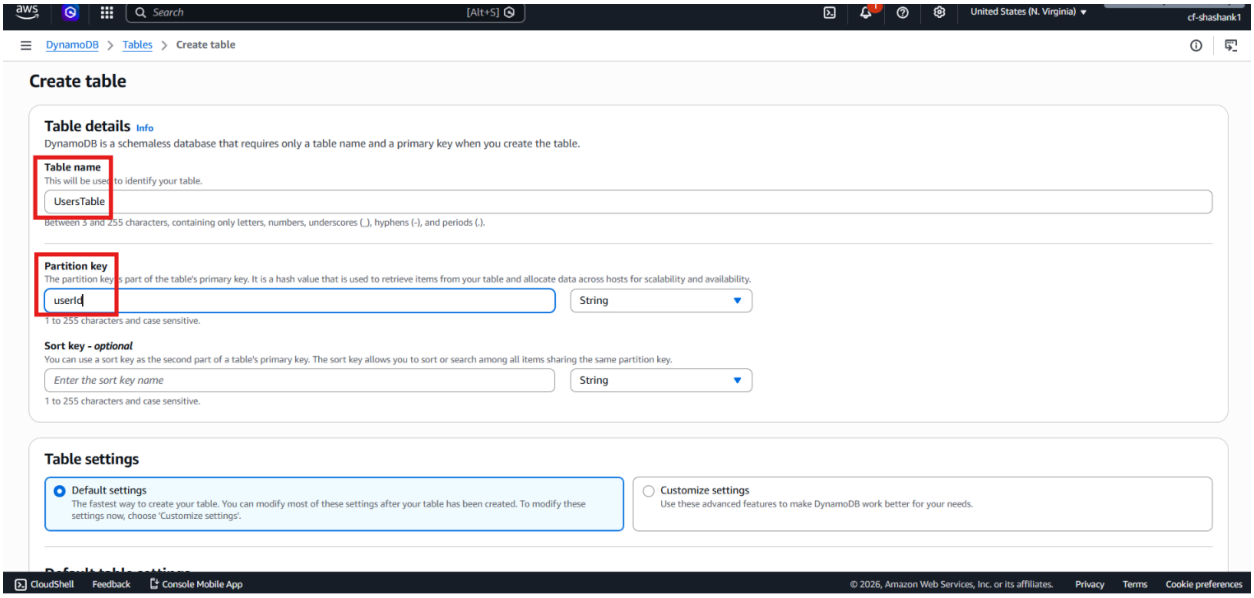
Lab Steps

Step 1 – Create the DynamoDB Backend Table

- Sign in to the AWS Management Console.
- Open **Amazon DynamoDB**.
- Click **Create table**.



- Enter the table name: UsersTable
- Set the partition key name to userId.
- Set the partition key type to String.
- Leave the sort key blank.



- Click **Create table**.

Step 2 – Understand the API Requirement

- Review the lab goal.
- Note that the front-end application cannot directly access DynamoDB.
- Understand that an API layer is needed for:
    - Creating a user
    - Fetching a user

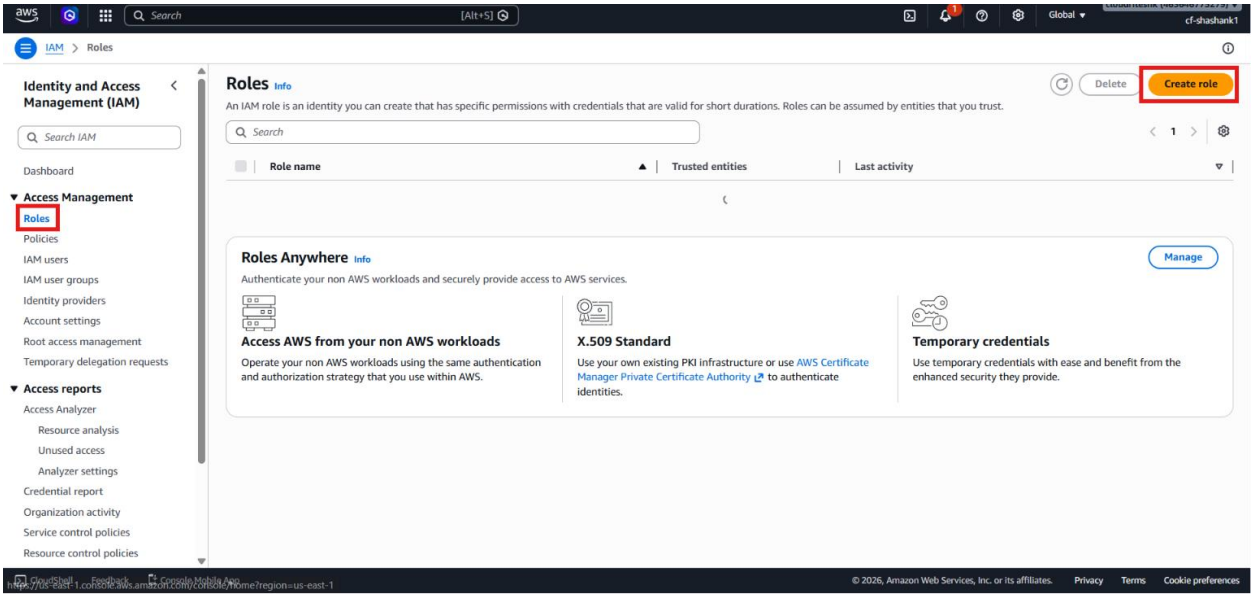
    - Deleting a user
- Understand that the API logic will be hosted in **AWS Lambda**.

Step 3 – Plan the Lambda Functions

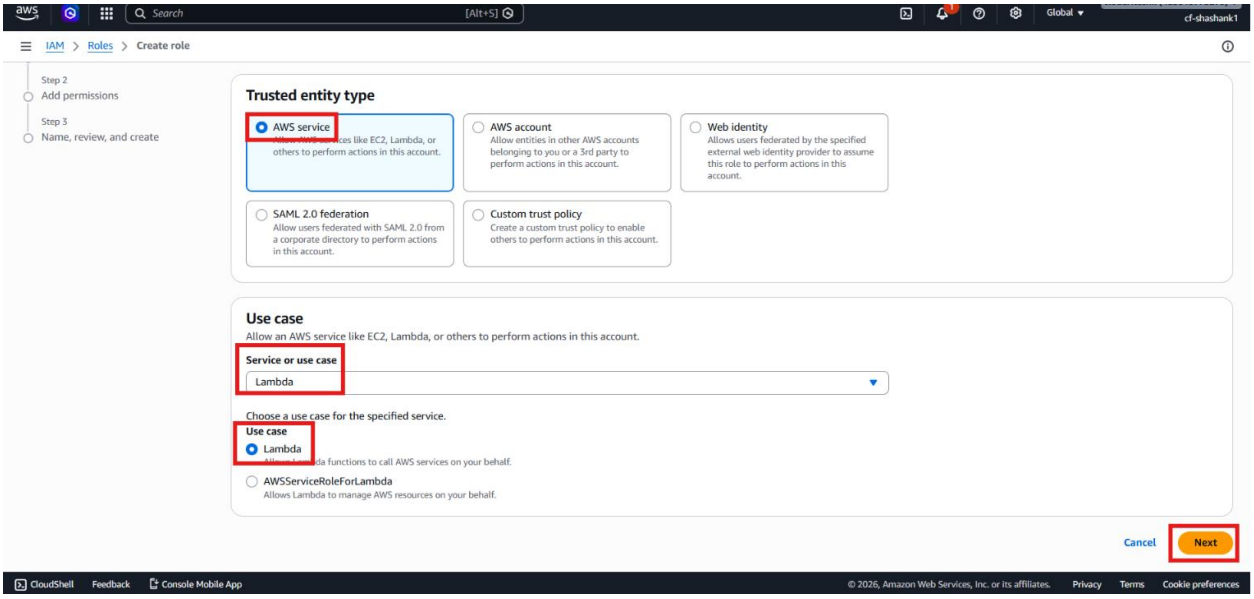
- Decide to create three separate Lambda functions:
    - createUserFunction
    - getUserFunction
    - deleteUserFunction
- Keep each function focused on one task.
- Follow separation of concerns for easier testing and maintenance.

Step 4 – Create the IAM Role for Lambda

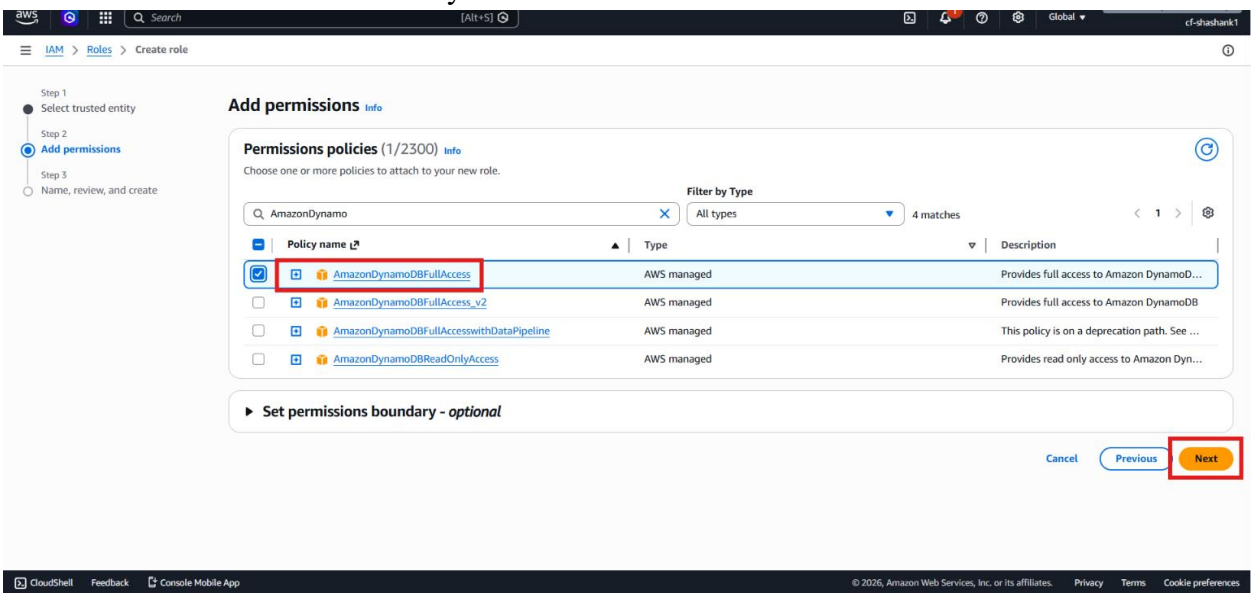
- Open **IAM → Roles**.
- Click **Create role**.



- Choose **AWS service** as the trusted entity type.
- Select **Lambda**.



- Attach **AmazonDynamoDBFullAccess**.
- Enter the role name: LambdaDynamoDBExecutionRole



- Create the role.
- Save this role for all three Lambda functions.

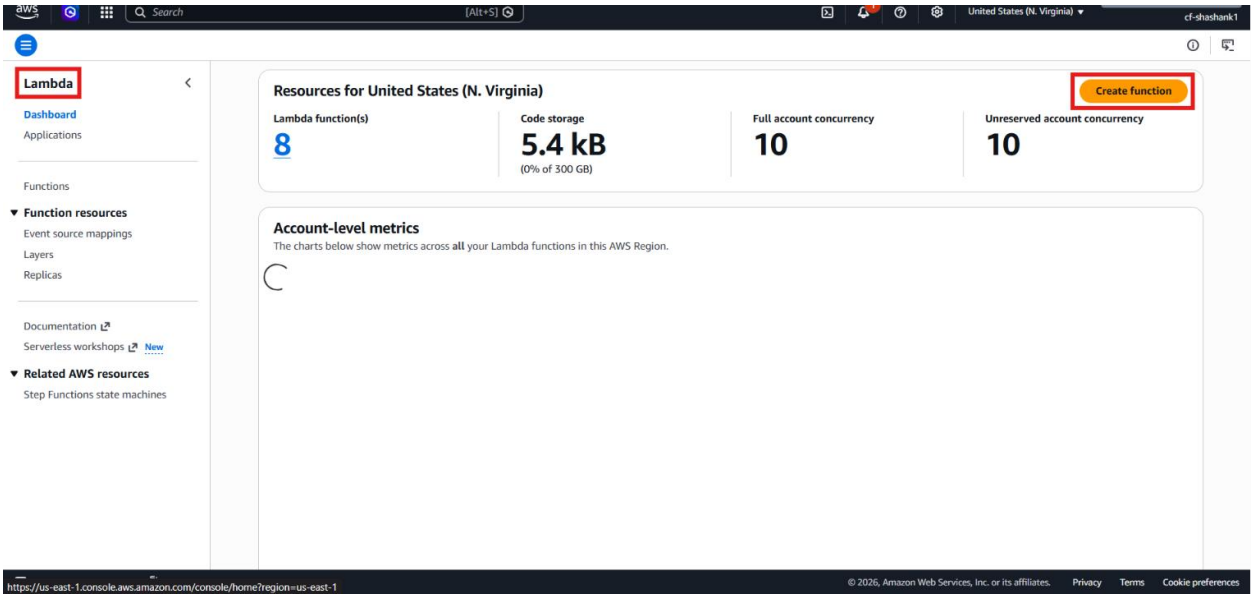
Step 5 – Confirm the Prerequisites Are Ready

- Verify that the **UsersTable** DynamoDB table exists.
- Verify that the **LambdaDynamoDBExecutionRole** has been created.
- Confirm that Lambda now has permission to talk to DynamoDB.
- Prepare to create the HTTP API in the next part of the lab.

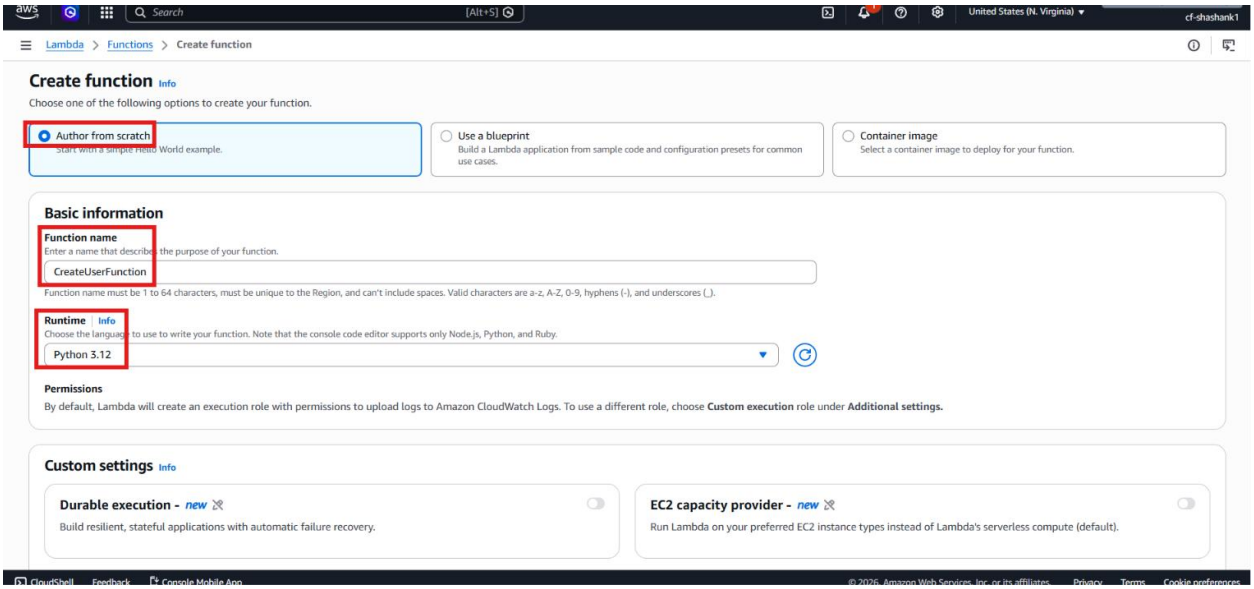
Step 6 – Create the First Lambda Function

- Sign in to the AWS Management Console.

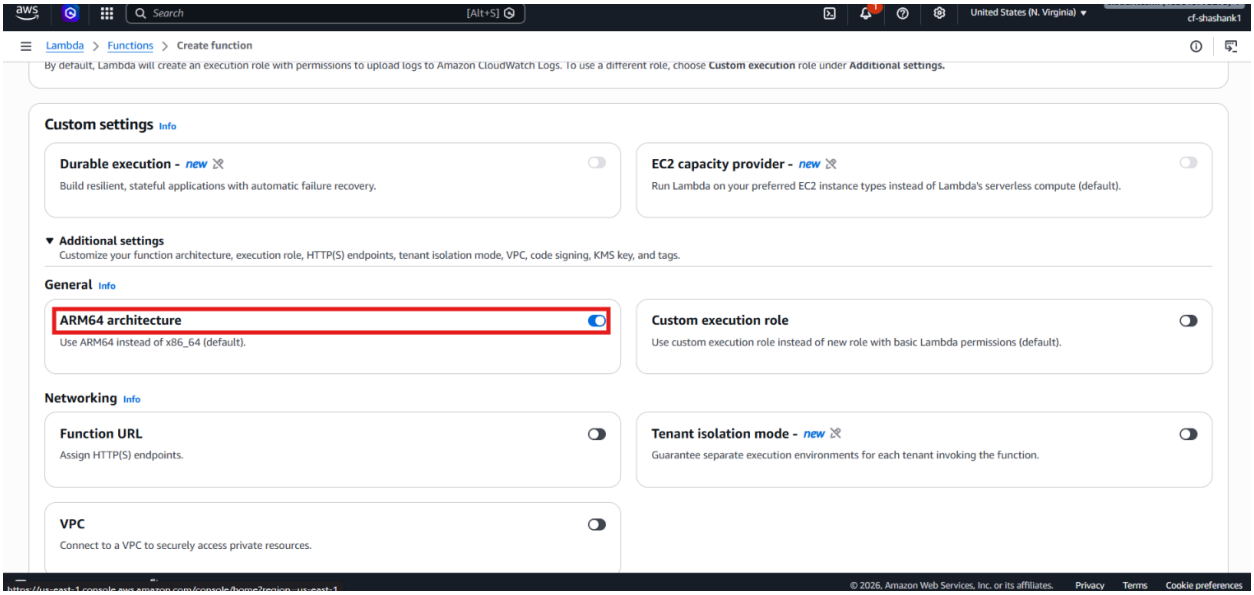
- Open **AWS Lambda**.
- Click **Create function**.



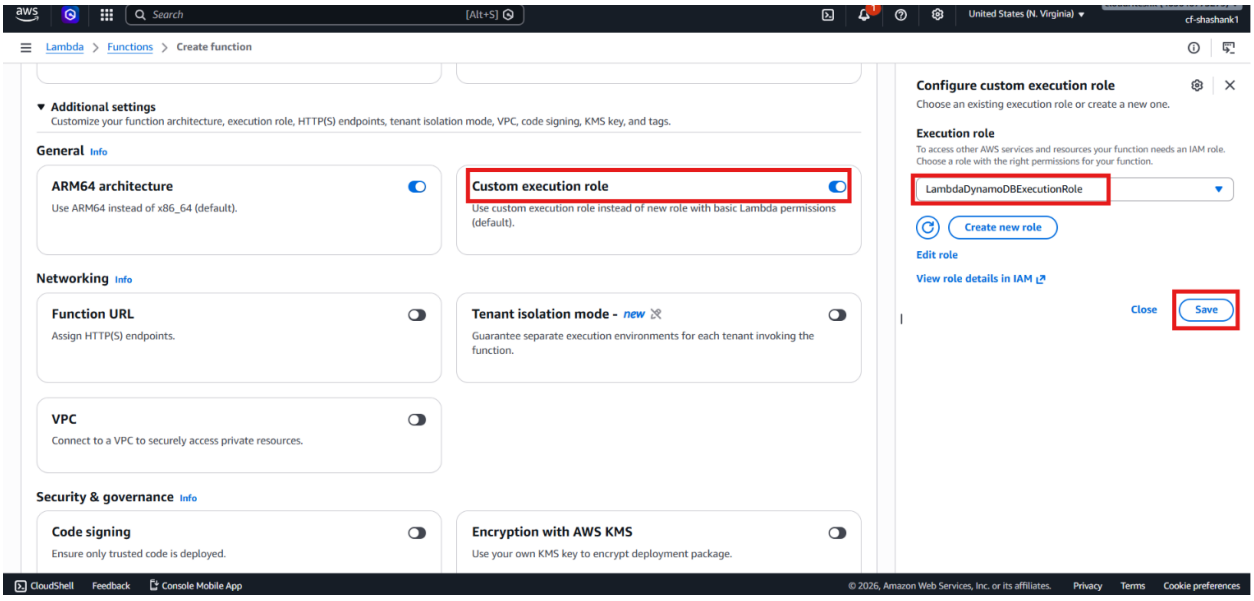
- Select **Author from scratch**.
- Enter the function name: CreateUserFunction
- Choose the runtime: Python 3.12



- Under Custom Settings → Additional Settings, enable **ARM64 architecture** to use **x86_64**.



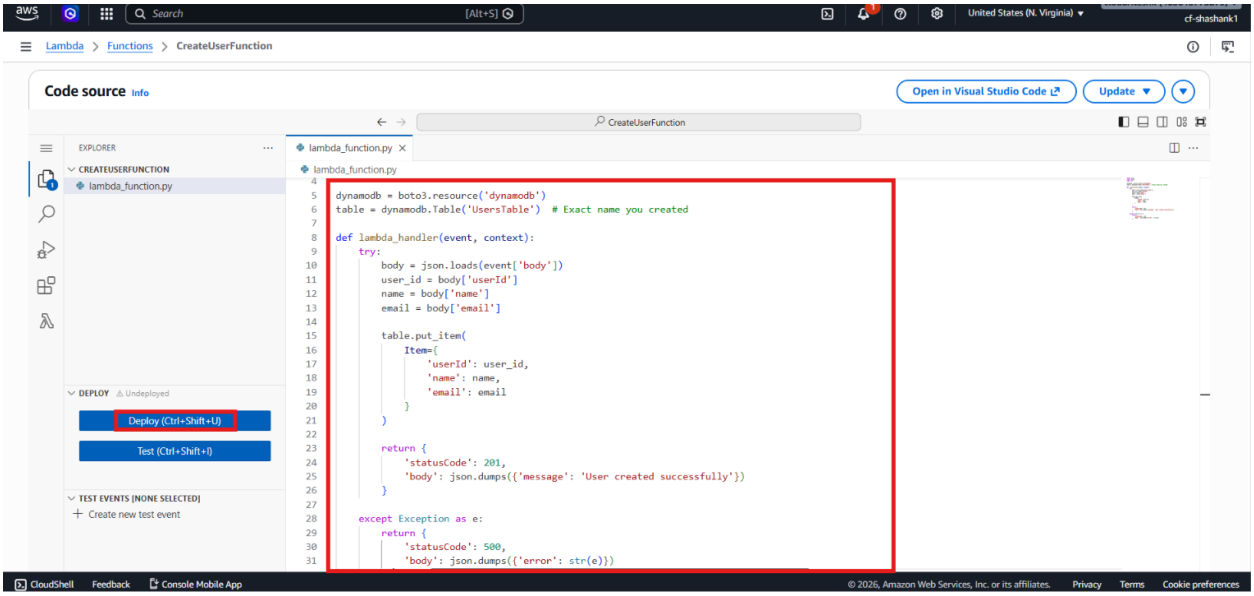
- Here Under **Additional Settings** click on **Custom Execution Role** and Select **LambdaDynamoDBExecutionRole**.



- Click **Create function**.

Step 7 – Add the Create User Code

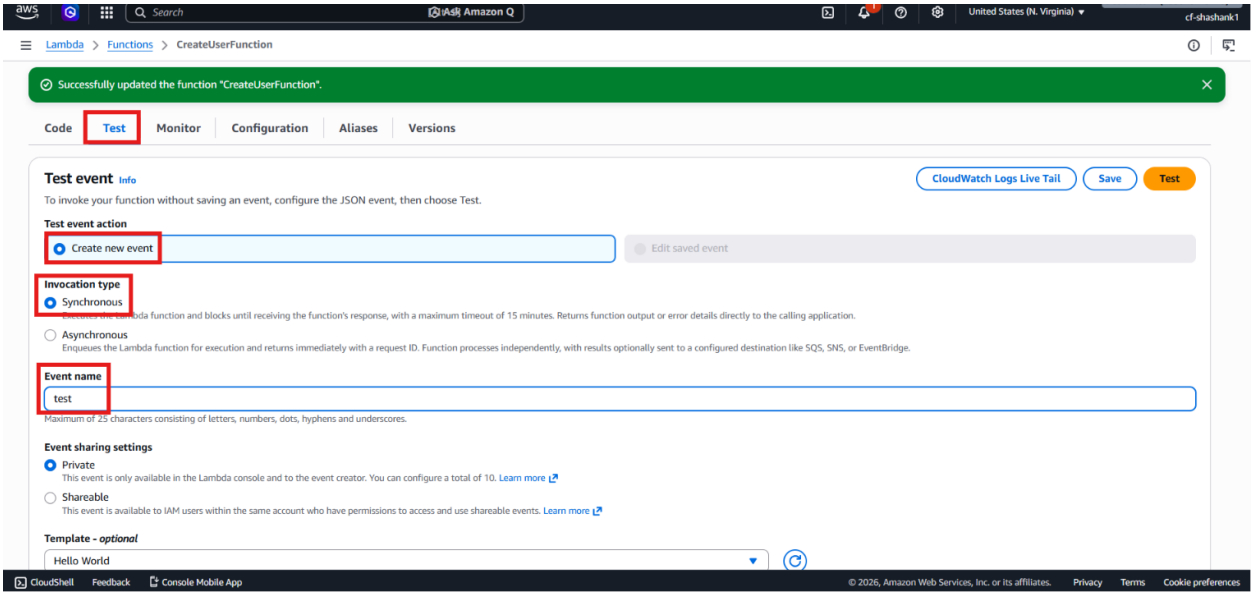
- Open the code editor.
- Replace the default handler code with the provided create-user Lambda code.
- Deploy the function.



- Confirm that the function was updated successfully.

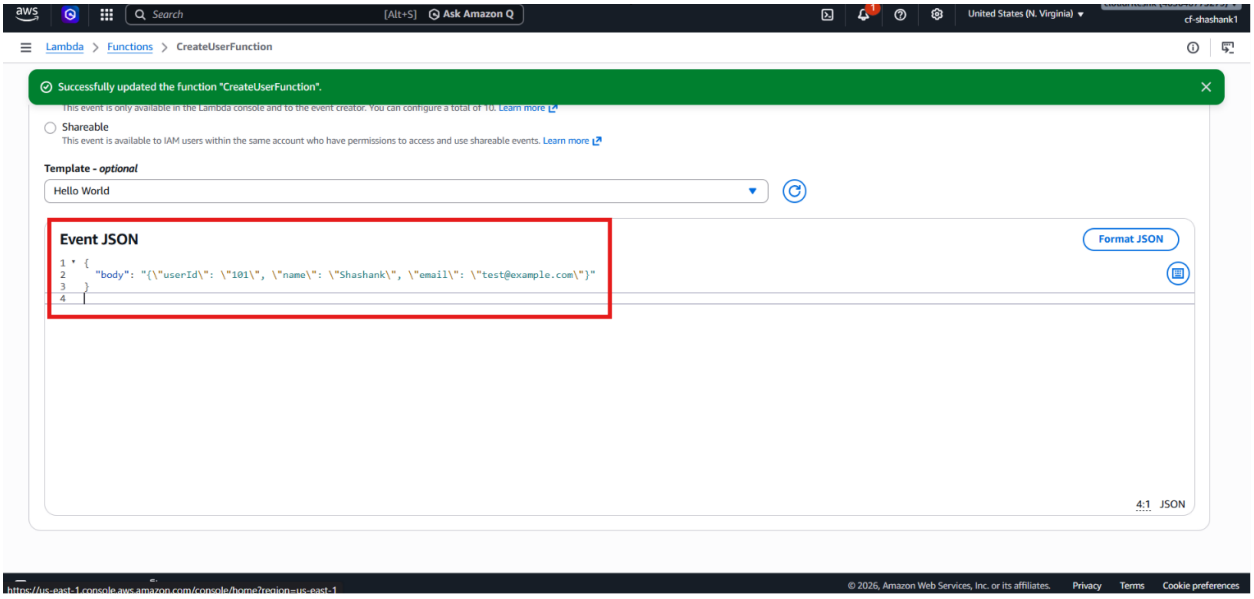
Step 8 – Test the Create User Function

- Open the **Test** tab.
- Click **Create new test event**.
- Enter a test event name.

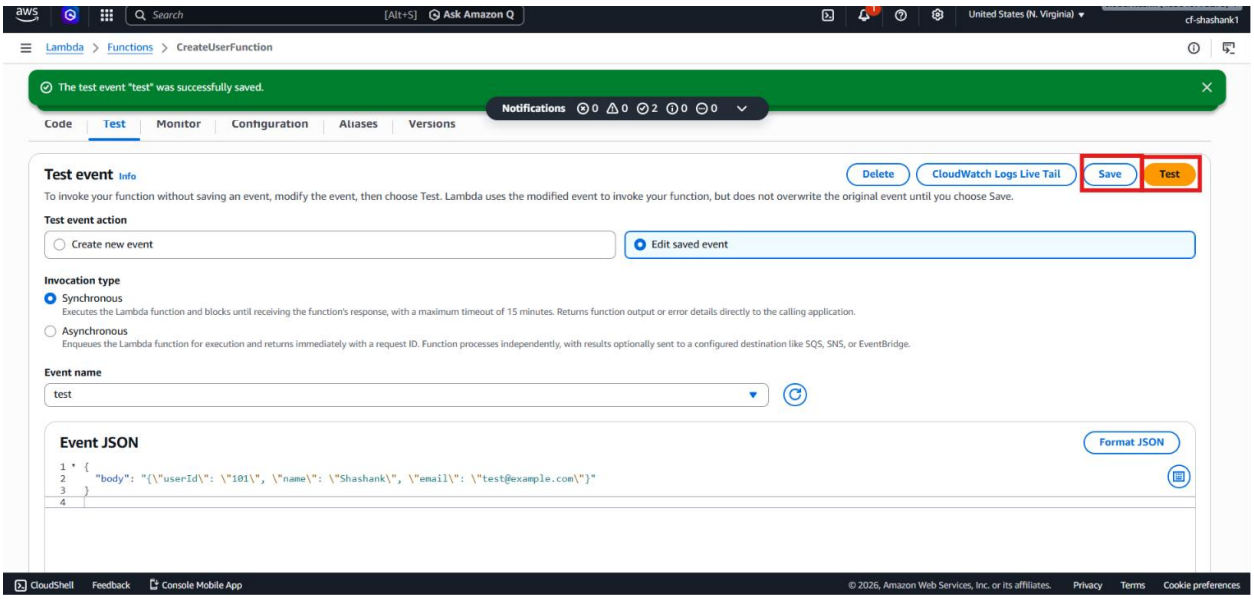


- Use an API Gateway–style payload with the body as a JSON string.

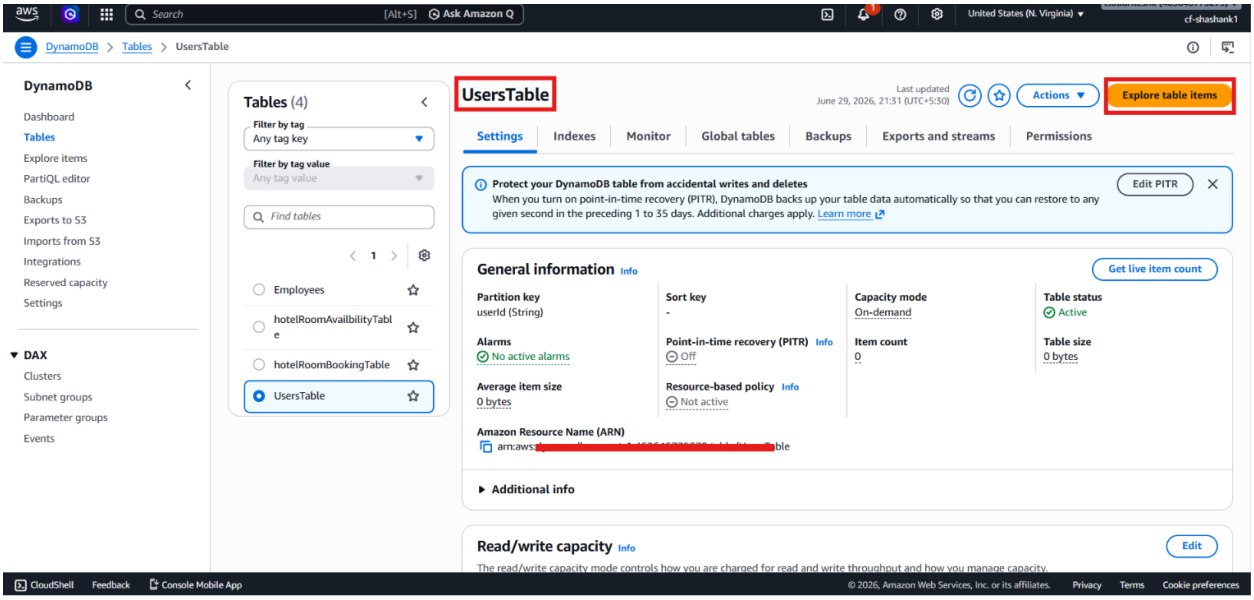
```
{
  "body": "{\"userId\": \"101\", \"name\": \"Shashank\", \"email\": \"test@example.com\"}"
}
```



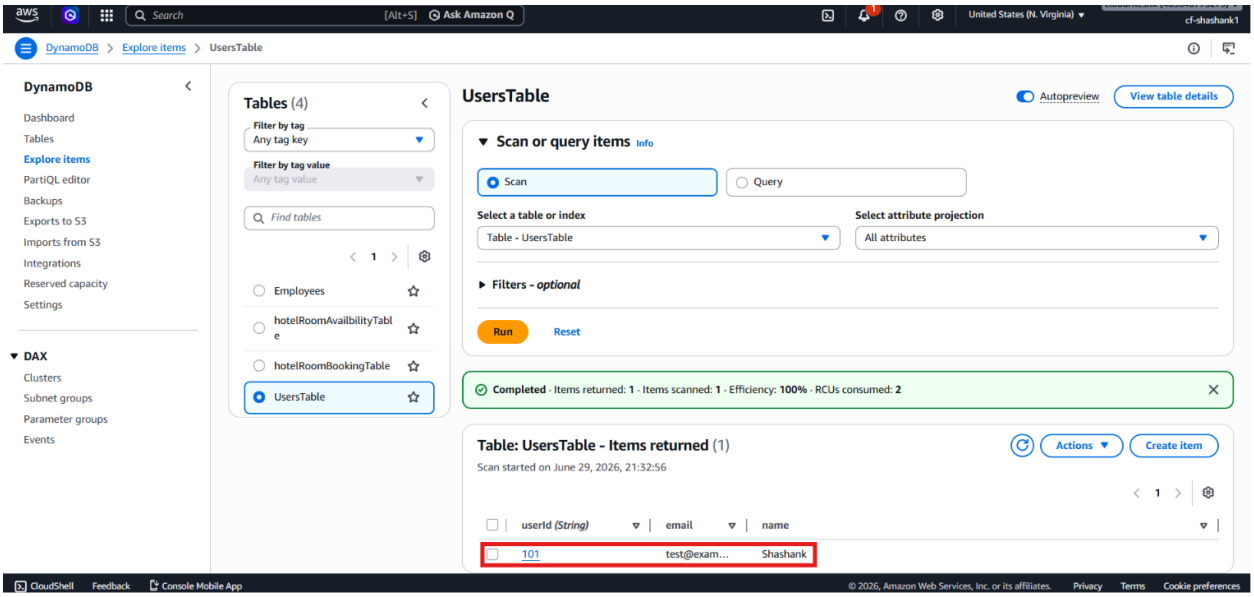
- Click **Save**.
- Click **Test**.



- Verify that the function returns a success status.
- Open **Amazon DynamoDB**.
- Go to **Tables → UsersTable**.
- Click **Explore table items**.

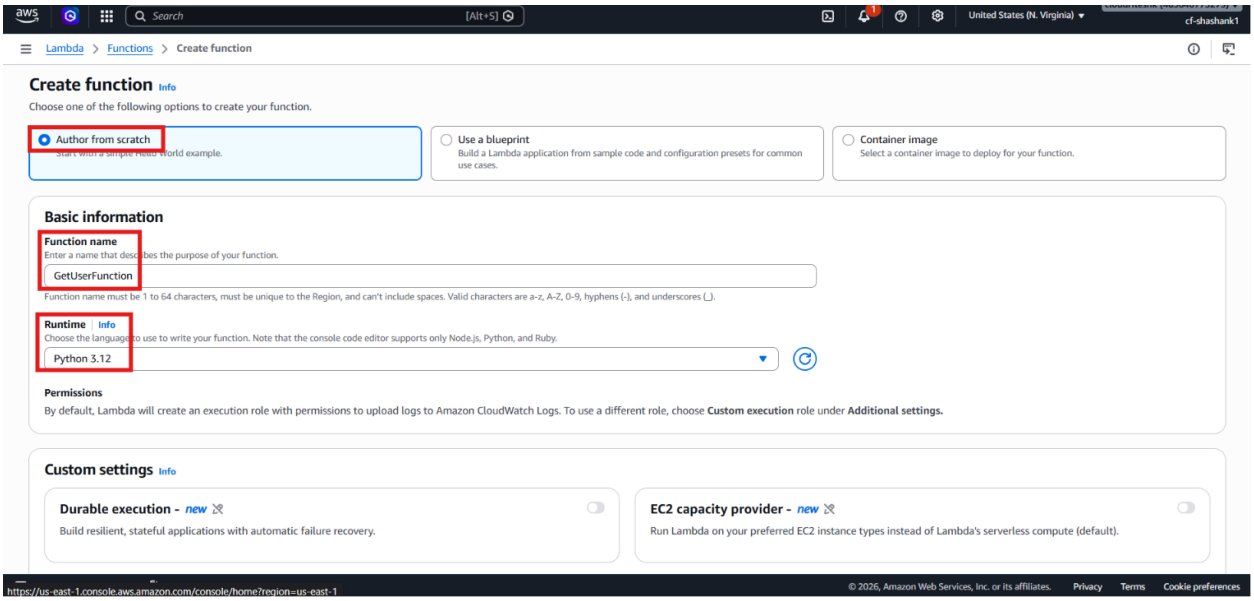


- Confirm that the new record was inserted.

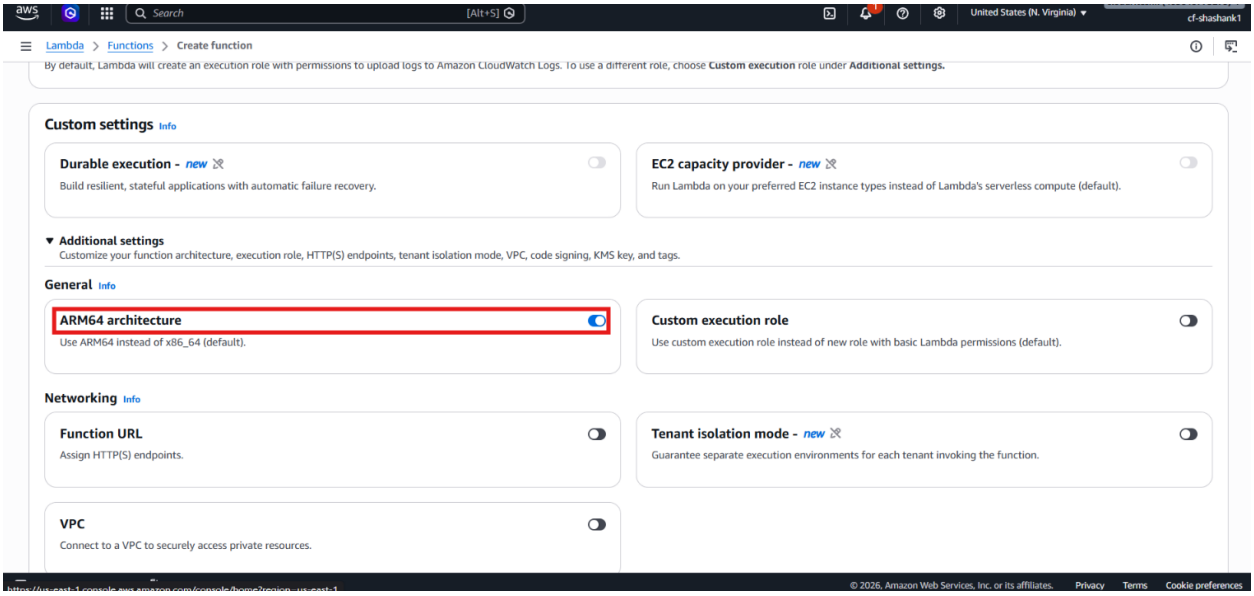


Step 9 – Create the Get User Function

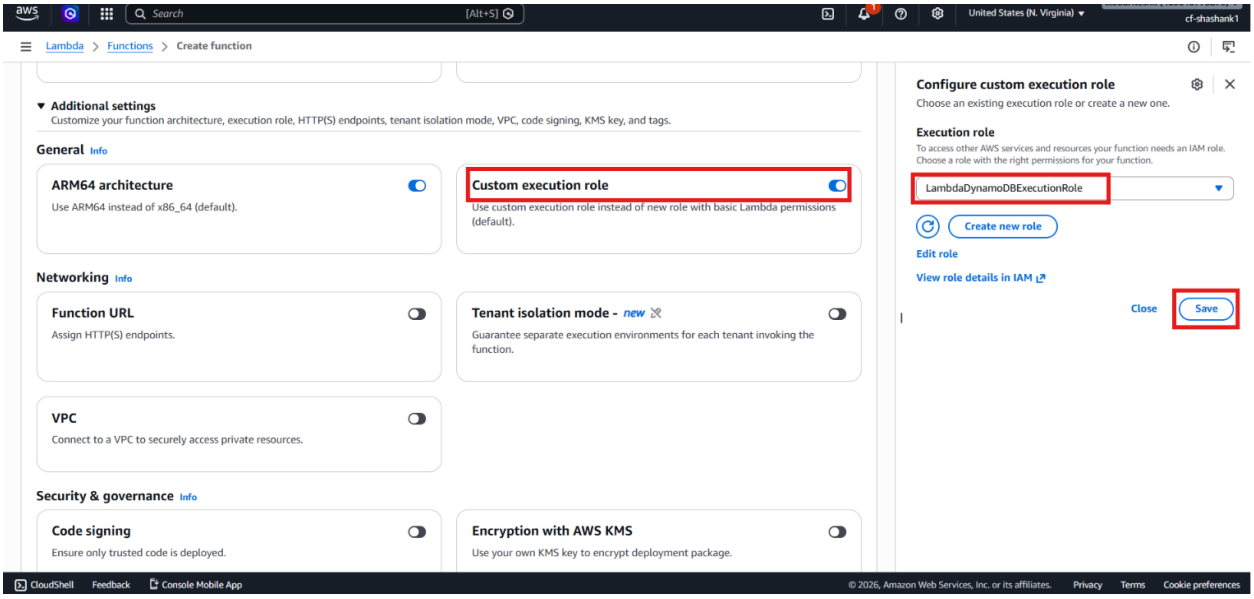
- Return to **AWS Lambda**.
- Click **Create function**.
- Choose **Author from scratch**.
- Enter the function name: GetUserFunction
- Choose **Python 3.12**.



- Under Custom Settings → Additional Settings, enable **ARM64 architecture** to use **x86_64**.



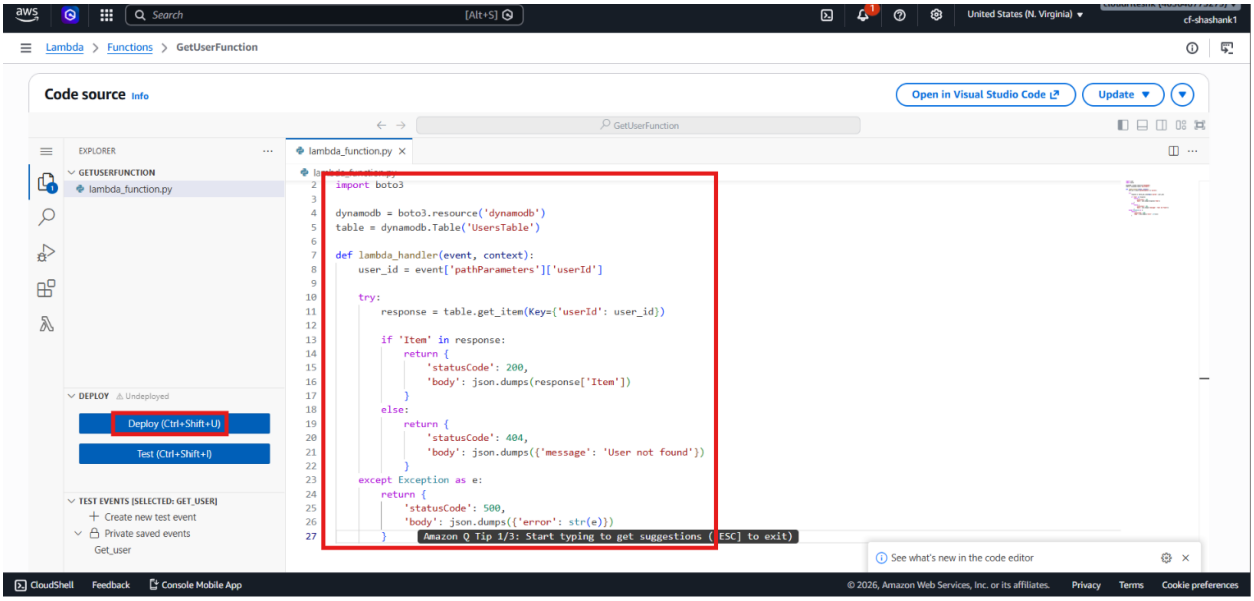
- Here Under **Additional Settings** click on **Custom Execution Role** and use the existing role **LambdaDynamoDBExecutionRole**.



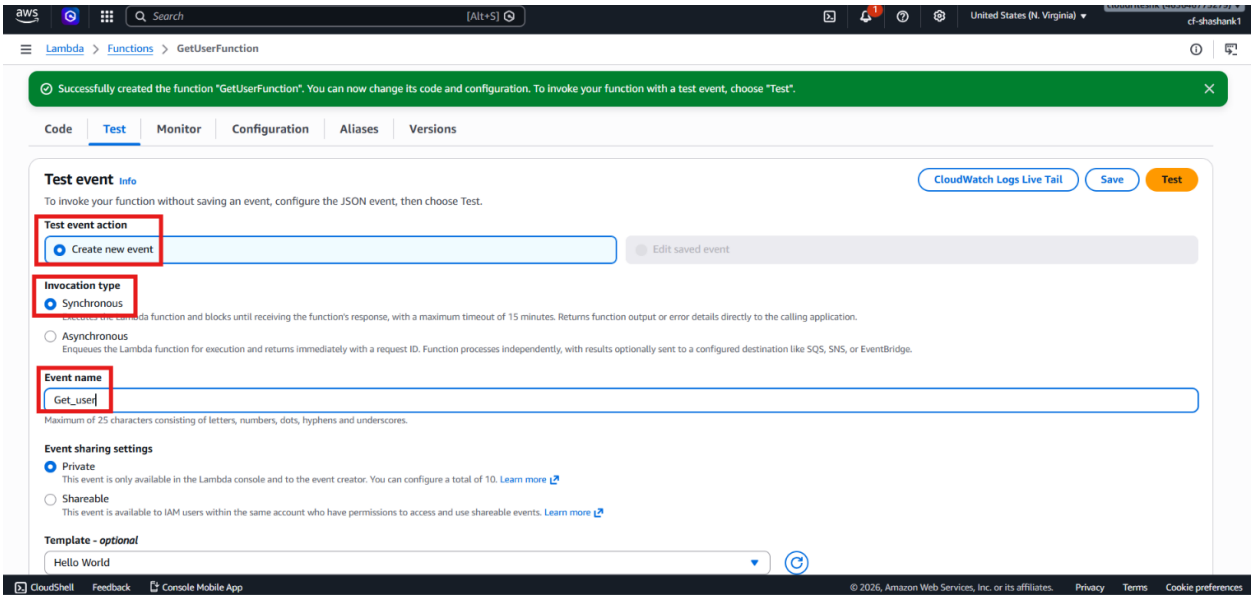
- Click **Create function**.

Step 10 – Add the Get User Code

- Replace the default code with the provided get_user Lambda code.
- Deploy the function.

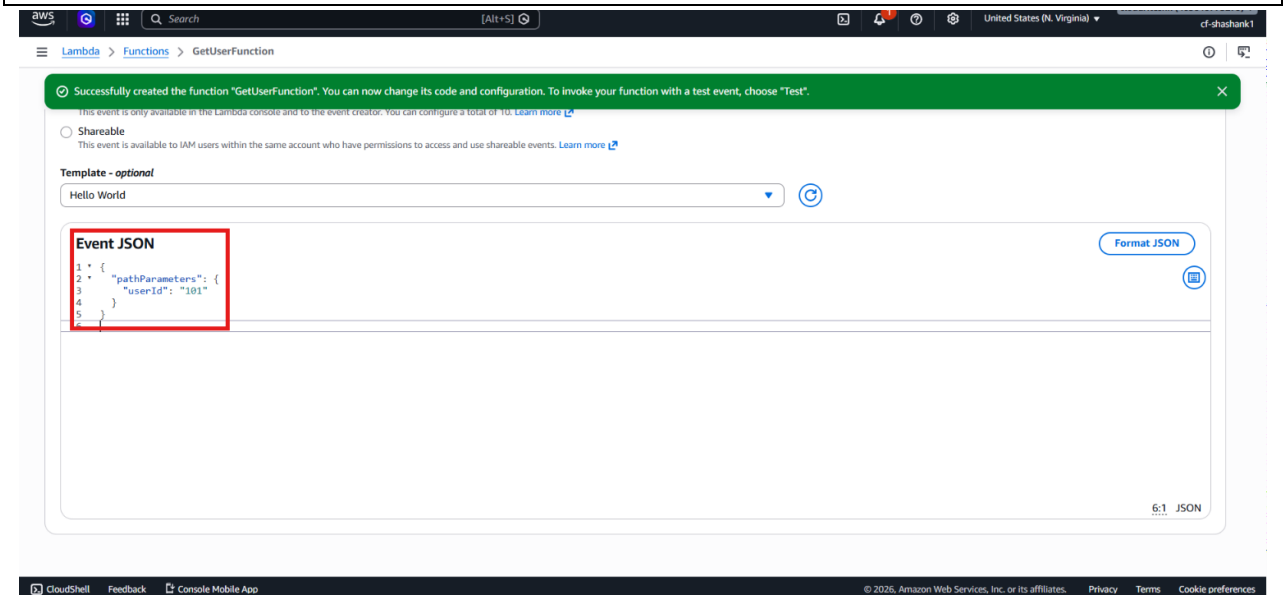


- Open the **Test** tab.

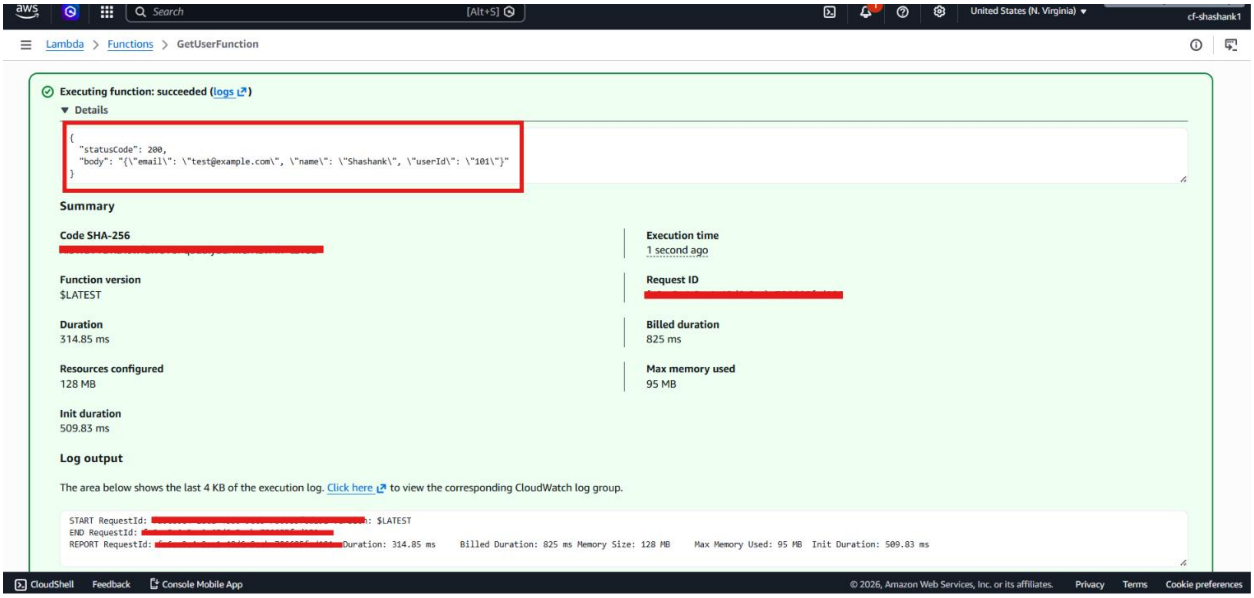


- Create a new test event using the path parameter structure.

```
{
  "pathParameters": {
    "userId": "101"
  }
}
```



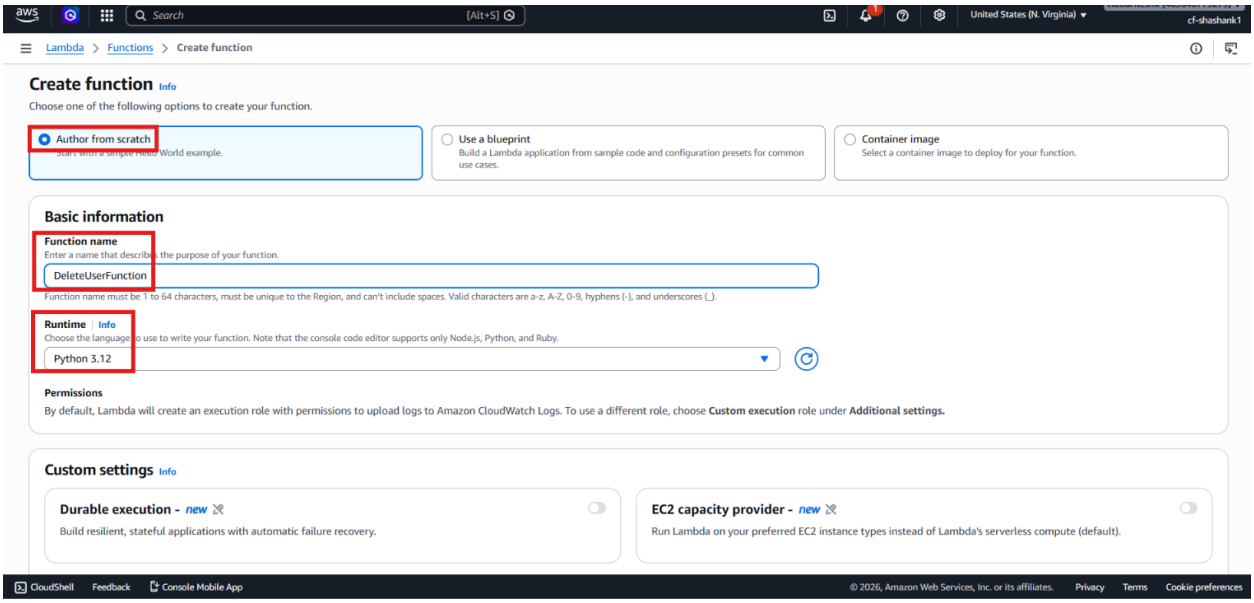
- Click **Save**.
- Click **Test**.



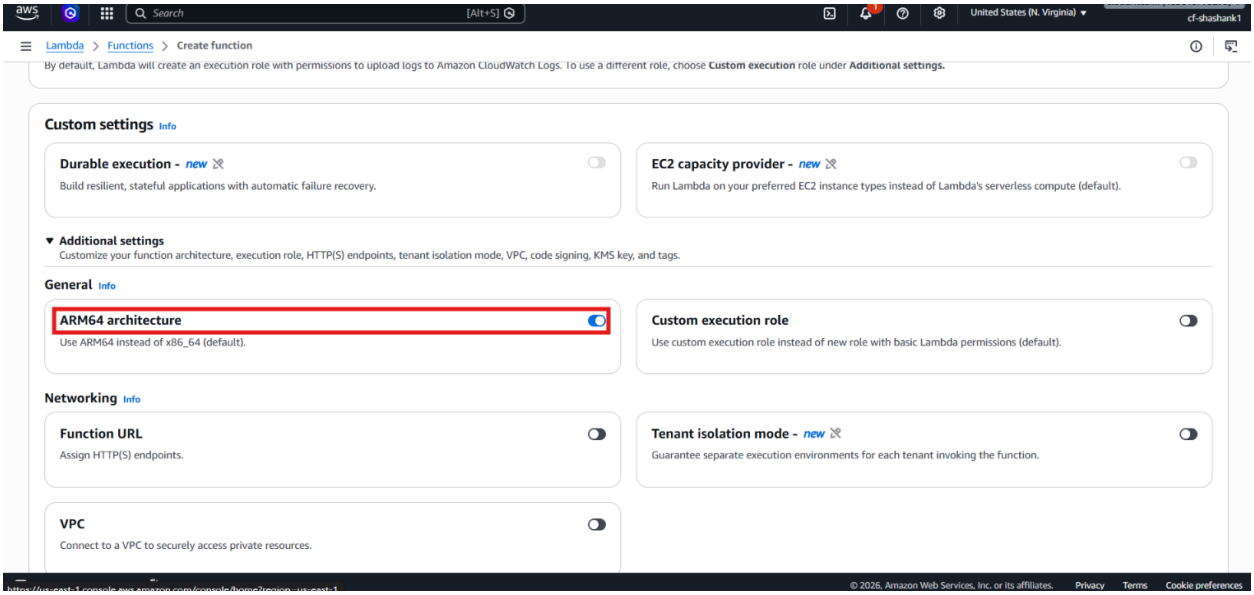
- Verify that the function returns the user details if the record exists.
- Change the userId to a missing value such as 102.
- Test again.
- Confirm that the function returns **User not found** for a missing record.

Step 11 – Create the Delete User Function

- Return to **AWS Lambda**.
- Click **Create function**.
- Choose **Author from scratch**.
- Enter the function name: DeleteUserFunction
- Choose **Python 3.12**.



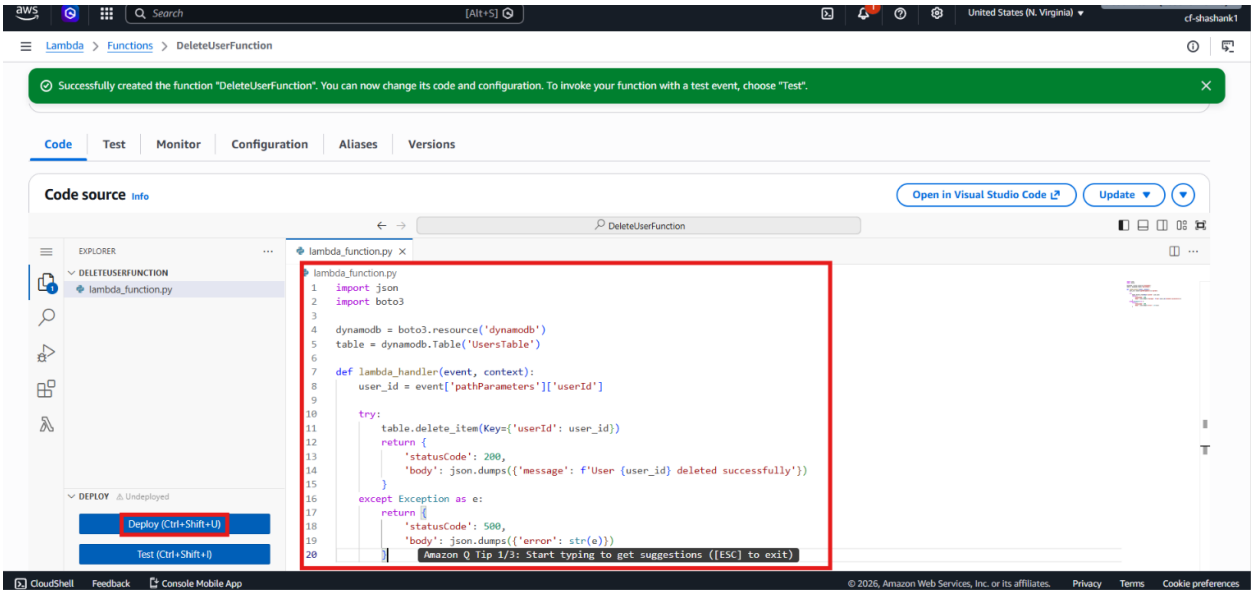
- Under Custom Settings → Additional Settings, enable **ARM64 architecture** to use **x86_64**.



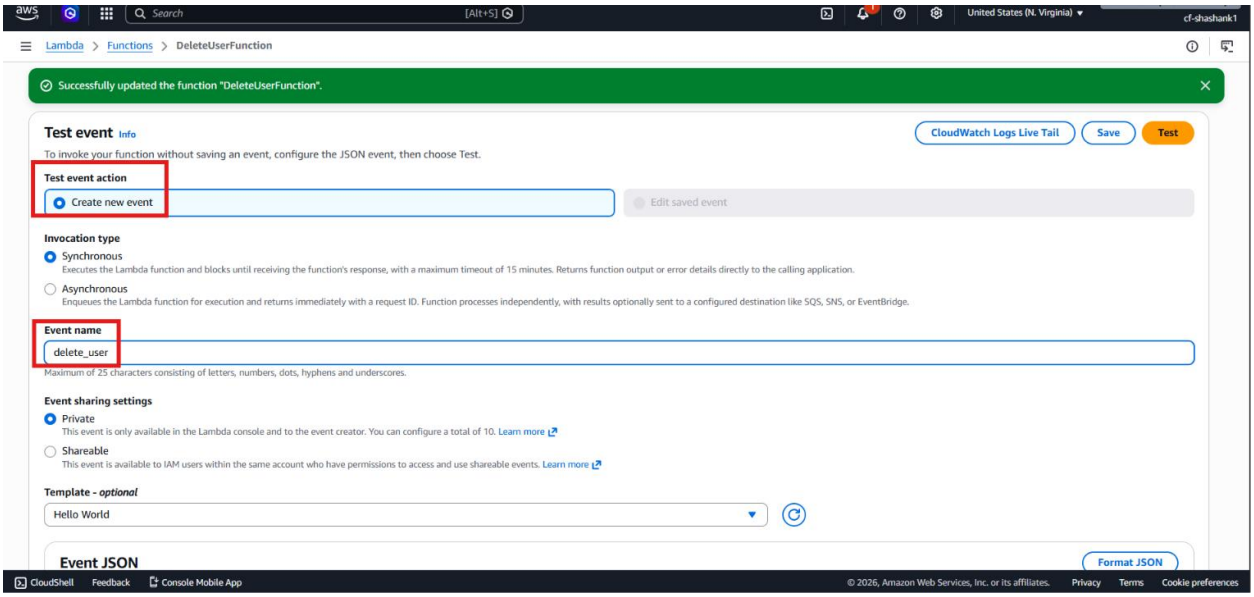
- Here Under **Additional Settings** click on **Custom Execution Role** and use the existing role **LambdaDynamoDBExecutionRole**.
- Click **Create function**.

Step 12 – Add the Delete User Code

- Replace the default code with the provided delete-user Lambda code.
- Deploy the function.

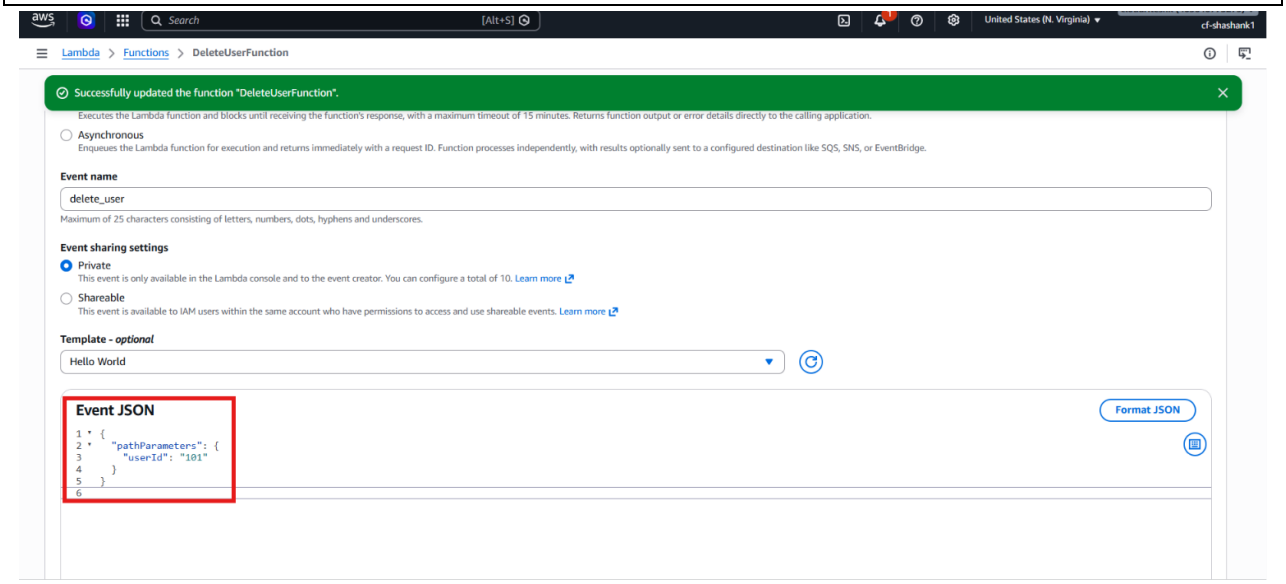


- Open the **Test** tab.

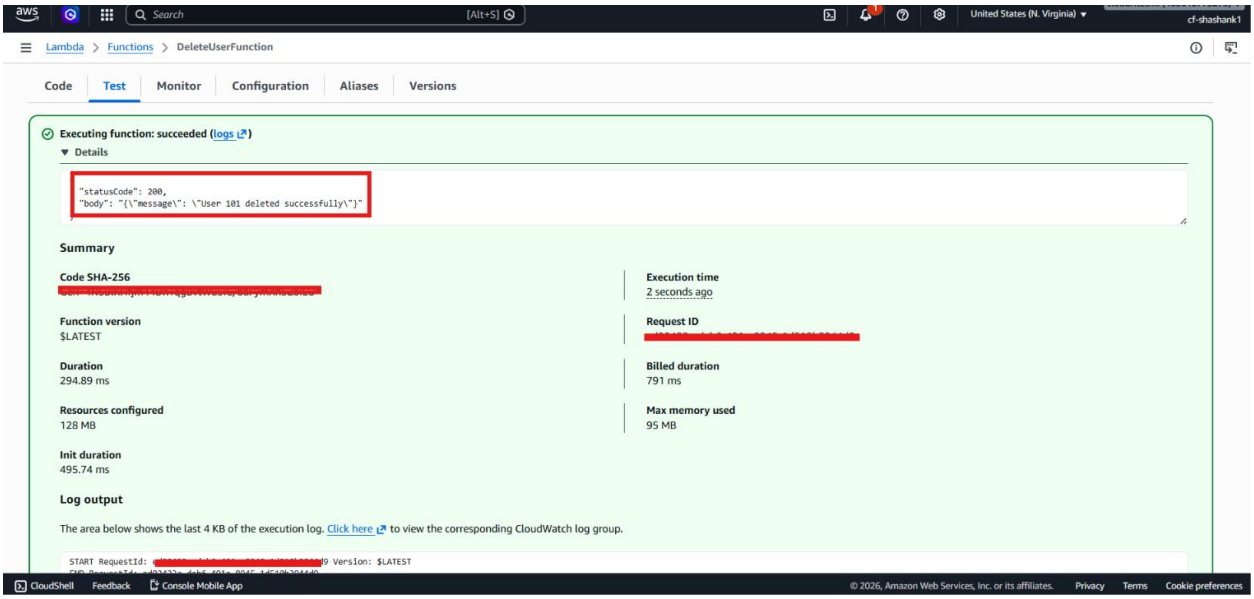


- Create a new test event using the same path parameter structure.

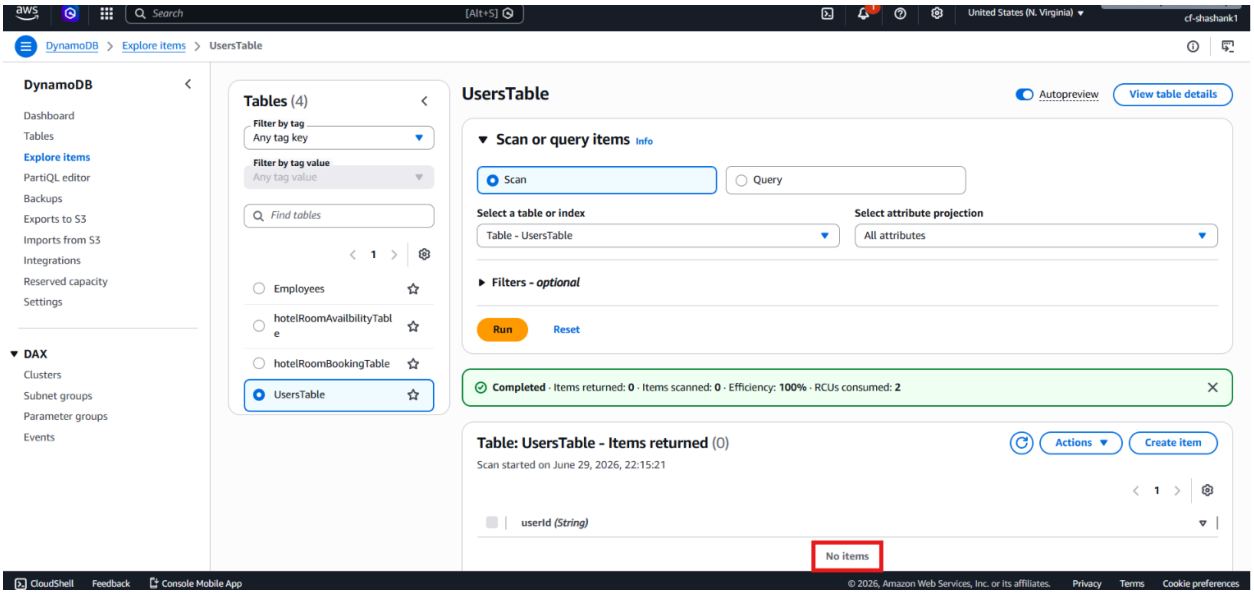
```
{
  "pathParameters": {
    "userId": "101"
  }
}
```



- Click **Save**.
- Click **Test**.



- Verify that the function returns a deletion success message.
- Open **Amazon DynamoDB**.
- Refresh **UsersTable**.



- Confirm that the record was removed.

Step 13 – Confirm the CRUD Backend Is Ready

- Verify that all three Lambda functions exist:
  - createUserFunction
  - getUserFunction
  - deleteUserFunction
- Confirm that all three functions use the same **IAM role**.

- Confirm that the DynamoDB table has been tested successfully with create, read, and delete operations.
- Prepare for **API Gateway** integration in the next part of the lab.

Verification

- Verified that the **UsersTable** was created successfully in **Amazon DynamoDB** with userId as the partition key.
- Verified that the **LambdaDynamoDBExecutionRole** was created and attached to all three **AWS Lambda** functions.
- Verified that **CreateUserFunction** successfully inserted a new user record into the DynamoDB table.
- Verified that **GetUserFunction** returned the correct user details for an existing userId.
- Verified that **GetUserFunction** returned **"User not found"** for a non-existent userId.
- Verified that **DeleteUserFunction** successfully removed the user record from the DynamoDB table.
- Verified that the DynamoDB table reflected the create and delete operations correctly.
- Verified that the CRUD backend was fully functional and ready for **Amazon API Gateway** integration.